Emergent Language Decoding: Level 1 Results

Headline numbers are **mean ± std over 3 training seeds** (0, 1, 2) on one dataset (`data_seed` 0), all on a laptop CPU. Setup and commands are in README.md; the spec is in BRIEF.md. Per-run tables: `results/summaries/modes*.csv` and `results/summaries/distractors*.csv`.

TL;DR

- **Gumbel works:** the default game (V=10, L=3, K=4, random distractors) reaches **0.92 ± 0.03** on test against 0.25 chance (seeds: 0.944, 0.938, 0.883). About 7 minutes per run on CPU.
- **Baselines sit at chance:** the no-communication listener gets 0.25 ± 0.01; replacing the trained speaker's message with random or fixed symbols gives 0.24 / 0.25. The message is used.
- **REINFORCE gets stuck:** 0.67 ± 0.06, with only 3-6 distinct messages in every seed.
- **The language encodes position and size, about half of shape, and little colour.** It is not compositional: no single symbol position carries shape or colour.
- **Generalisation to unseen shape-colour pairs is good in easy games** (held-out 0.92 vs test 0.92) but **opens a gap in the hardest game** (K=8, hard distractors: 0.68 vs 0.81).
- Runs reproduce exactly: the same seed gave identical numbers in two different sweeps.

1. Main results (K=4, random distractors, 3 seeds)

From `results/summaries/modes_agg.csv`.

	Gumbel	REINFORCE	No communication
Test accuracy (chance 0.25)	**0.922 ± 0.034**	0.667 ± 0.056	0.248 ± 0.006
Held-out vs test distractors	0.919 ± 0.034	0.639 ± 0.087	0.252 ± 0.039
Held-out, own games (see §4)	0.950 ± 0.041	0.657 ± 0.041	0.249 ± 0.004
Test, random message	0.238	0.246	0.259
Test, fixed message	0.252	0.260	0.248
Topographic similarity (test)	0.32 ± 0.08	0.47 ± 0.07	-
Distinct messages (test)	112 ± 68	4.3 ± 1.5	1

All configs run 12 epochs. REINFORCE uses the tuned settings (entropy bonus 0.001, baseline decay 0.9), which are now the defaults in `train.py`.

Topsim caveat: REINFORCE has *higher* topsim than Gumbel while communicating far less. With 3-6 messages that mostly track position, message distance lines up with one attribute and looks "structured". Always read topsim alongside accuracy and lexicon size.

2. What the language encodes

Normalised mutual information between the **whole message** and each attribute on the test split (0 = tells you nothing, 1 = fully determines it), mean over 3 seeds:

Config	Position	Size	Shape	Colour
Gumbel, K=4 random	0.95	0.93	0.52	0.31
Gumbel, K=4 hard	0.96	0.41	0.31	0.34
Gumbel, K=8 random	1.00	0.98	0.55	0.23
REINFORCE, K=4 random	0.92	0.12	0.03	0.12

The best *single* message position carries at most ~0.1 of shape or colour; the shape information lives in combinations of symbols. A concrete example from seed 0: the large red circle on the left and the large blue square on the left both get the message `7-7-7`.

Other properties of the Gumbel code (seed 0, K=4 random):

- **Synonyms:** a given attribute combination gets its most common message only ~47% of the time. The speaker likely reacts to the small pixel jitter.
- **Hard distractors make the code more entangled, not more compositional.** Messages still pin down the full combination (NMI with the combination 0.46-0.73), but line up less with single attributes, and topsim stays at ~0.31.

An earlier version of this file said that colour and shape "must sit in combinations of symbols" because accuracy was 95%. That was wrong for colour: most colour information is simply not sent (see §3 for why the game allows this).

3. Why accuracy plateaus where it does

With m equally likely messages, each random distractor has the target's message with probability 1/m, and the listener must guess among ties. Expected accuracy for K=4:

Speaker describes	Messages	Predicted	Observed
position only	3	0.60	0.60-0.61 (stuck Gumbel tuning runs, REINFORCE seed 1)
position + size	6	0.78	0.77-0.81 (stuck Gumbel tuning runs)
position + size + shape	24	0.95	0.94 (best Gumbel seeds)

So the Gumbel plateau of ~0.94 is about what a speaker that ignores colour entirely would get. With random distractors, colour only matters when the distractor matches on everything else, which is rare.

Getting Gumbel past the position-only trap (seed 0 tuning)

With temperature 1.0 and batch 32 the first run plateaued at 0.61 with 3 messages. The fix, config only, was **temperature 2.0 and batch 128** (now the defaults). Variants tried (6-epoch runs):

Variant	Test acc	Messages
Original (temp 1.0, batch 32)	0.60	3-4
Temperature 0.5	0.61	3
Learning rate 3e-4	0.77	6
BatchNorm in the CNN	0.77	6
Entropy bonus on the Gumbel speaker	0.69-0.77	11-19
Temperature 2.0	0.81	27
Batch 128	0.81	15
Temperature 2.0 + batch 128 (20 epochs)	0.95	180

Even with the fix, seed 2 of the default config only reached 0.88 with 34 messages, so the trap is weakened, not gone.

4. Held-out evaluation, corrected

The `heldout` split contains only the 4 held-out (shape, colour) pairs (red square, magenta square, green triangle, magenta triangle with `data_seed` 0), and its games draw distractors from that split only. That is an easier game (fewer look-alikes), which is why held-out accuracy came out *above* test. `final.json` now also reports `heldout_vs_test`: held-out targets with distractors from the test split, the same distractor pool as the test number. Use that one for generalisation claims.

5. Game difficulty (Gumbel, 3 seeds)

From `results/summaries/distractors_agg.csv`.

K	Chance	Random distractors	Hard distractors	Hard, held-out vs test
2	0.50	0.951 ± 0.011	0.914 ± 0.018	0.918
4	0.25	0.922 ± 0.034	0.893 ± 0.045	0.874
8	0.125	0.891 ± 0.013	0.814 ± 0.082	**0.684**

Harder games lower accuracy and widen the seed spread (K=8 hard: 0.74 / 0.80 / 0.90), but stay far above chance. Generalisation to unseen pairs holds in easy games and breaks down in the hardest one.

6. REINFORCE

Seed 0 tuning (20 epochs each):

Setting	Test acc	Messages
Entropy 0.01, baseline decay 0.99	0.45	2
No entropy bonus	0.61	3
Entropy 0.001	0.61	3
Entropy 0.001, lr 3e-4	0.61	3
Entropy 0.001, baseline decay 0.9	0.80	6

The 0.80 run was not saved and did not reproduce in the 3-seed sweep (12 epochs: 0.70, 0.60, 0.70), so quote **0.67 ± 0.06**. The collapsed 2-message run split colours into {blue, magenta} vs {green, red, yellow}, i.e. one bit about the blue channel. REINFORCE does not only describe position. Likely next fixes (code changes, need a team decision): per-batch advantage normalisation and a separate speaker learning rate.

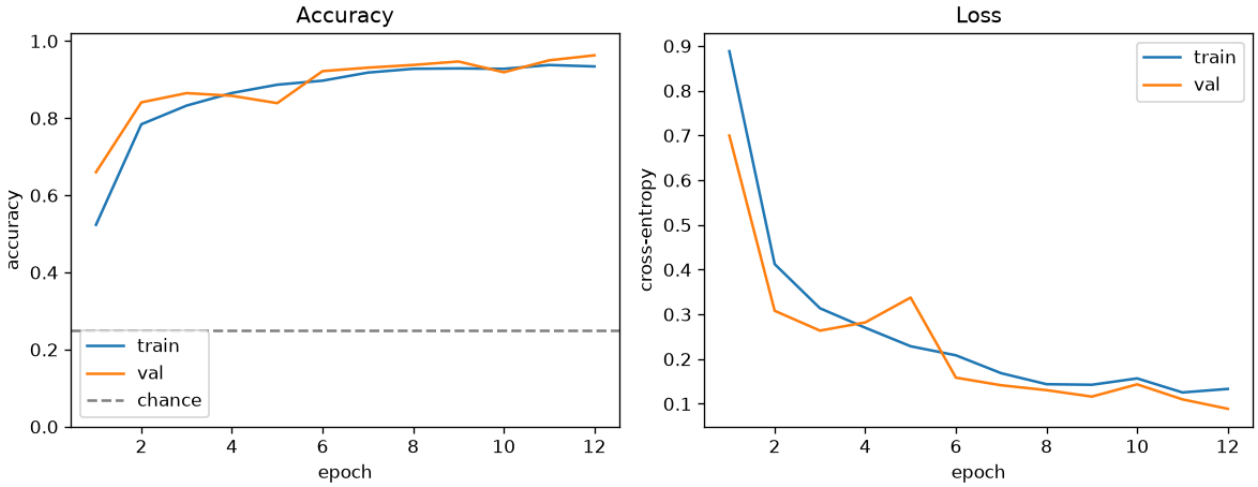
7. Design choices worth knowing

- **Held-out split:** 4 of the 20 (shape, colour) pairs (20%), chosen with `data_seed`. Every shape and every colour still appears in training with other partners.
- `seed` **vs** `data_seed`**:** `seed` controls training; `data_seed` controls the images and held-out pairs. All training seeds are evaluated on the same data and the same eval games.
- **Distractors never share the target's exact combination.** Otherwise the game could only be won by reading pixel jitter.
- **Jitter is sub-pixel**, so test images are new instances, not pixel copies of training images.
- `train_time_sec` **is wall-clock time** and includes the laptop sleeping. One run reported 6,923 s although its epochs took ~33 s each; use the per-epoch `elapsed_sec` in `metrics.csv`.
- **Reproducibility:** identical results need the same seed *and* the same number of CPU threads.

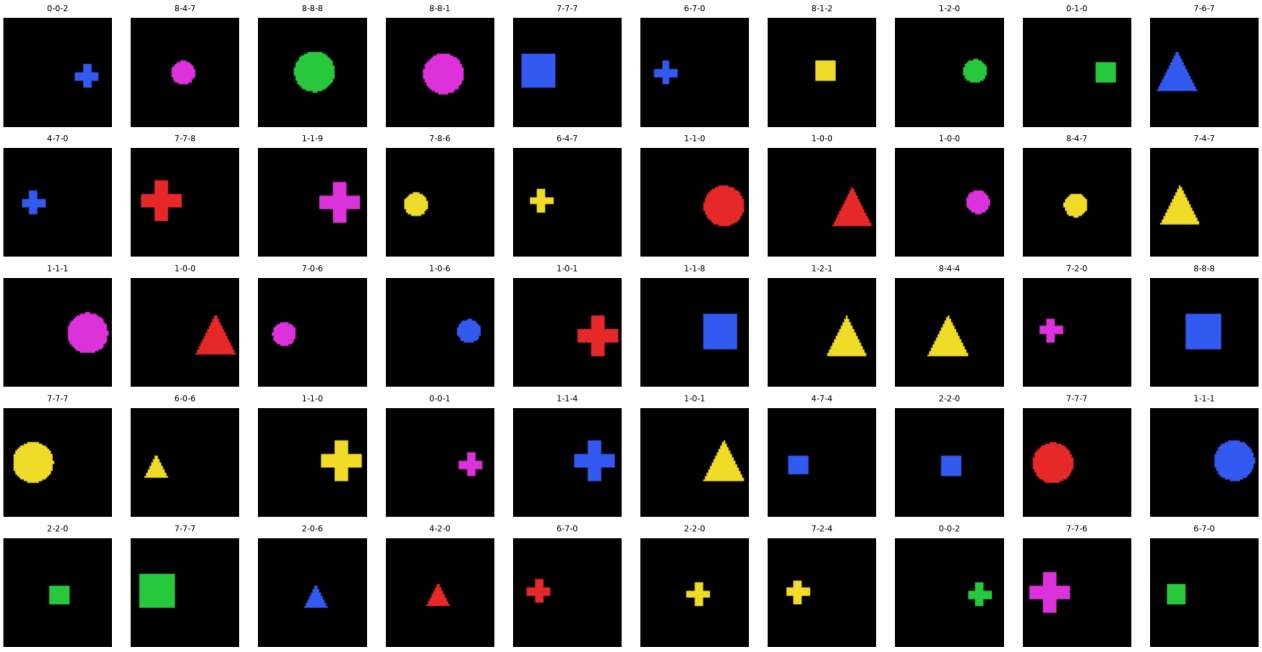
8. Open items

1. **Vocabulary × length sweep:** `python scripts/sweep.py configs/sweep_vocab_len.yaml` (27 runs, about 3 h on CPU; run one process per seed with `--seeds N`, then once without `--seeds` to write the summary).
2. **Make colour matter:** a game whose distractors differ only in colour or shape, to test whether the code becomes compositional when the game needs it. Not in the spec yet.
3. **REINFORCE:** try the code changes in §6, or report the gap as it is.
4. **More dataset seeds** for the headline numbers.

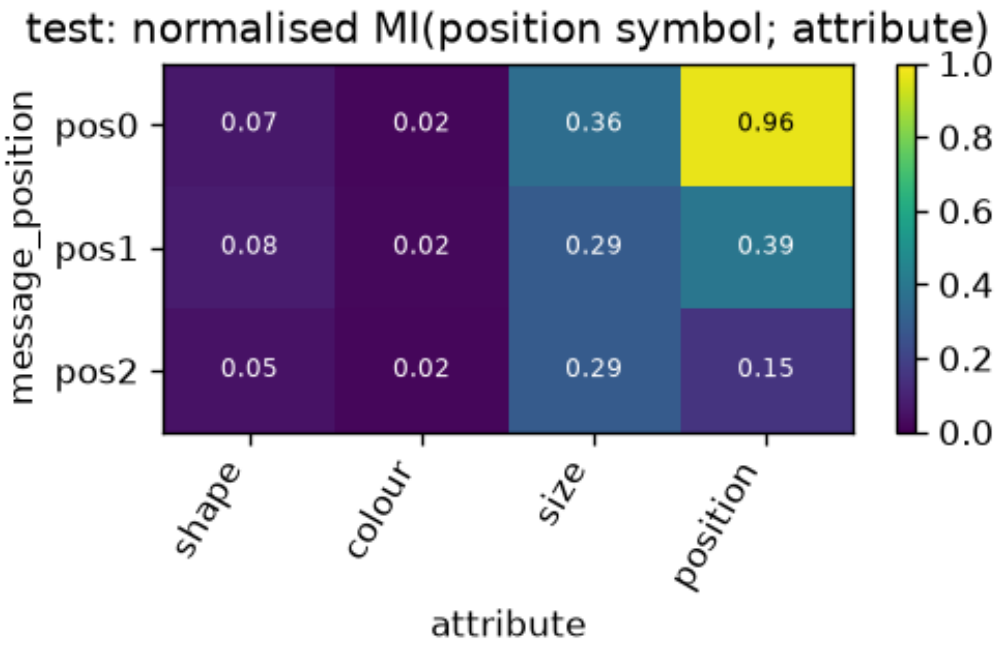
Figures (Gumbel, K=4 random distractors, seed 0)



Training curves.



Test-split targets, each with the speaker's message shown above it.



Mutual information between each message position and each attribute (test).